Отчёт по отладке приложения
Кофанов Арсений Михайлович

Шаг 1. Анализ до запуска.

Был проанализирован весь код проекта, заранее были исправлены некоторые ошибки, которые не критичны, но не соответствуют стандартам.

- В файле зависимостей библиотек выставлены версии библиотек, убраны повторы и излишний код.

```
1     - fastapi
2     - uvicorn[standard]
3     - sqlalchemy>=2.0
4     - asyncpg
5     - alembic
6     - pydantic
7     - pydantic-settings
8     - httpx
9     - apscheduler
10    - fastapi==999.0.0; python_version < "3.8"
     1+ uvicorn[standard]==0.41.0
     2+ sqlalchemy==2.0.48
     3+ asyncpg==0.31.0
     4+ alembic==1.18.4
     5+ pydantic==2.12.5
     6+ pydantic-settings==2.13.1
     7+ httpx==0.28.1
     8+ apscheduler==3.11.2
     9+ fastapi==0.135.0
11   10
```

requirements.txt до / после.

- Дублирование функции get_session. Убраны функции из parse.py и vacancies.py и перенесена в session.py.

- В файле external.py у класса ExternalVacanciesResponse указан излишний алиас, дублирующий смысл.

До: item_count: int = Field(alias="item_count")

После: item_count: int

- database_url в config.py имел параметр validation_alias с опечаткой, без настроенного .env код бы не хаупустилмя. Так же стандартное значение database_url имеет иную базу, по сравнению с тем, что указано в docker_compose для контейнера db.

До: database_url: str = Field("postgresql+asyncpg://postgres:postgres@db:5432/postgres_typo", validation_alias="DATABSE_URL")

После: database_url: str = Field("postgresql+asyncpg://postgres:postgres@db:5432/postgres", validation_alias="DATABASE_URL").

Шаг 2. Исправление ошибок запуска.

Был запущен docker compose up build, запущены контейнеры. После запуска было выявлено 2 проблемы:

- Задача запускается чаще, чем указано в переменной окружения. Найдено места настройки задач и установлен множитель.

До: seconds=settings.parse_schedule_minutes

После: seconds=settings.parse_schedule_minutes * 60

- При выполнении задачи парсинга возникала ошибка получения данных для вакансий без города. В схеме указано, что город — опциальный параметр. Проблема была исправлена, добавив условие.

До: "city_name": item.city.name.strip()

После: "city_name": item.city.name.strip() if item.city else None

Шаг 3. Общий анализ.

После успешного запуска и анализа работы были выявлен проблемы, которые могут ухудшить работу сервиса в дальнейшем.

- Метод on_event не рекомендуется к использованию и будет упразднён, изменён на реализацию через lifespan.

```
17  @asynccontextmanager
18  async def lifespan(app: FastAPI):
19      global _scheduler
20
21      logger.info("Запуск приложения")
22      await _run_parse_job()
23
24      _scheduler = create_scheduler(_run_parse_job)
25      _scheduler.start()
26
27      yield
28
29      logger.info("Остановка приложения")
30      if _scheduler:
31          _scheduler.shutdown(wait=False)
32
33
34  app = FastAPI(title="Selectel Vacancies API",
35                lifespan=lifespan
36               )
```

Lifespan

- Убраны стандартные варианты переменных для контейнера с postgres и заменены на переменные окружения. В конфиге переписана реализация получения ссылки для подключения. Обусловлено безопасностью и удобством работы.

```
1
2   services:
3     db:
4       image: postgres:16
5       environment:
 -        POSTGRES_USER: postgres
 -        POSTGRES_PASSWORD: postgres
 -        POSTGRES_DB: postgres
6+        POSTGRES_USER: ${POSTGRES_USER:-postgres}
7+        POSTGRES_PASSWORD: $
 +        {POSTGRES_PASSWORD:-postgres}
8+        POSTGRES_DB: ${POSTGRES_DB:-postgres}
```

docker_compose.yml

```python
    postgres_user: str
    postgres_password: str
    postgres_db: str

    database_url: str = ""

    log_level: str = "INFO"
    parse_schedule_minutes: int = 5

    def model_post_init(self, __context):
        self.database_url = f'postgresql+asyncpg://{self.postgres_user}:{self.postgres_password}@db:5432/{self.postgres_db}'

```

config.py

- Исправлена ошибка типизации у get_session.

```python

async def get_session() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_maker() as session:
        yield session

```

session.py

- parse_and_store создавал сессию, но не закрывал её. Может вызвать проблемы спустя некоторое время работы.

До: client = httpx.AsyncClient(timeout=timeout)

После: async with httpx.AsyncClient(timeout=timeout) as client:

- Эндпоинт создания вакансии, при наличии вакансии с переданным id в базе, возвращает 200, что противоречит логике status_code. Изменено на 409. Тут нарушение контракта.

```python
        if existing:
            return JSONResponse(
                status_code=status.HTTP_200_OK,
                content={"detail": "Vacancy with external_id already exists"},
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="Vacancy with external_id already exists"
            )
```

vacancies.py

- В функции upsert_external_vacancies несколько проблем.

Убран риск keyerror

Убран баг, при котором при external_id = 0 поиск по этому пункту не учитывался.

Убран лишний запрос в базу и защита от дубликатов external_id

Изменений много, они доступны по ссылке https://github.com/Rimuwu/selectest-api/commit/7e2da5f16734063c5a99ed82629ea372773fac09

Итог.

Была произведена отладка сервиса по парсингу вакансий. Был исправлен и улучшен код, проверены все эндпоинты и взаимодействие с базой.

При выполнении задания был использован ИИ для самопроверки, уточнения деталей.

Код и весь ход работы доступен по ссылке - https://github.com/Rimuwu/selectest-api